

# 어셈블리프로그램 설계 및 실습

## 과제N(숫자 입력) 보고서



|       |               |
|-------|---------------|
| 담당교수  | 교수님           |
| 실습분반  | 목요일 (1분반)     |
| 소속    | 컴퓨터정보공학부      |
| 학번·성명 | 이호정           |
| 제출일   | 2024년 11월 13일 |

### 문제 1번

#### 1. Problem Statement

이 과제의 전반적인 목적은 재귀 함수와 스택 사용을 통한 팩토리얼 계산을 구현해 내고, 이를 arm 코드로 작성할 수 있는 능력을 키우기 위해서 이다. 특히, ARM 어셈블리 기본 명령어의 이해와 활용 능력과, 스택 및 메모리 관리 능력, 재귀 함수 구현 능력, 레지스터와 메모리 조작 능력, 성능 분석 및 최적화 능력을 키우기 위한 과제이다.

그 중 2번 문제는 second operand의 시프트 연산을 이용하여 팩토리얼을 계산하는 문제로, second operand를 시프트 연산 하여 얻을 수 있는 장점에 대해 생각해 볼 수 있는 문제이다.

#### 2. Method

second operand 의 시프트 연산을 이용하여 값을 누적하는 방식으로 접근할 수 있다.

1. 계산할 값  $n$ 을 초기에 부여준다. 이 문제의 경우에는 10
2. 해당  $n$  을 하나씩 줄여가면서 팩토리얼 계산을 해준다.
3. 이 뜻은 먼저  $10 \rightarrow 10 \times 9 \rightarrow 10 \times 9 \times 8$  이런식으로 구현 한다는 뜻이다.
4. 물론 늘려가면서도 가능하다
5. 그러기 위해서 이전값을 받아올 레지스터와 그 레지스터를 시프트 연산하고 덧셈 혹은 뺄셈 하여서  $m * r0$  가 되도록 구현한다. 이 과정을 시프트 연산으로 수행 하는 것이다. 두번째 피연산자에 LSL #1 을 하면 2배 인 것이고 +1 을 하기 위해선 ADD 명령어를 추가로 해주면 된다.

#### 3. Result

작성한 과제 코드의 예상 결과, 실행 결과 등을 figure와 함께 작성합니다. 어셈블리프로그래밍 수업에서는 muVision에 있는 debugging 기능을 활용하여 레지스터, 메모리의 값의 변화를 첨부합니다.

디버깅을 하면서 값이 누적되는지 확인한다,

$10 \rightarrow 10 \times 9 \rightarrow 10 \times 9 \times 8$  의 값이 레지스터에 쌓이는 것을 확인 할 수있다.

즉, 레지스터의 저장되는 값이  $10 \rightarrow 90 \rightarrow 720 \rightarrow \dots \rightarrow 3628800$ 으로 누적되는 것을 확인 할 수 있다.

## 어셈블리프로그램 설계 및 실습

C:\Users\suzy1\Desktop\HW1\pro1.uvproj - μVision

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Registers

| Register        | Value             |
|-----------------|-------------------|
| <b>Current</b>  |                   |
| R0              | 0x00375F00        |
| R1              | 0x001BAF80        |
| R2              | 0x40000000        |
| R3              | 0x00000000        |
| R4              | 0x00000000        |
| R5              | 0x00000000        |
| R6              | 0x00000000        |
| R7              | 0x00000000        |
| R8              | 0x00000000        |
| R9              | 0x00000000        |
| R10             | 0x00000000        |
| R11             | 0x00000000        |
| R12             | 0x00000000        |
| R13 (SP)        | 0x00000000        |
| R14 (LR)        | 0x00000000        |
| <b>R15 (PC)</b> | <b>0x00000070</b> |
| CPSR            | 0x000000D3        |
| SPSR            | 0x00000000        |
| User/System     |                   |
| Fast Interrupt  |                   |
| Interrupt       |                   |
| Supervisor      |                   |
| Abort           |                   |
| Undefined       |                   |
| Internal        |                   |

Disassembly

```

54:      MOV R7, #1
0x00000070: E3A07001 MOV     R7, #0x00000001
55:      MOV R0, #0
0x00000074: E3A00000 MOV     R0, #0x00000000

```

problem1.s

```

49:      ADD R0,R1,R1 ; R0=R1+R1=2*R1=3628800
50:
51:      LDR R2, =0x40000000 ;Save to memory address 0x400
52:      STR R0, [R2]
53:
54:      MOV R7, #1
55:      MOV R0, #0
56:      SVC #0
57:
58:      END
59:

```

Memory 1

Address: 0x40000000

|             |                                                 |
|-------------|-------------------------------------------------|
| 0x40000000: | 00 5F 37 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| 0x40000015: | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| 0x4000002A: | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| 0x4000003F: | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| 0x40000054: | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| 0x40000069: | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| 0x4000007E: | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |

Command

MAP 0xFFFF0000, 0xFFFFFFFF READ

### → 전체 결과 화면

Registers

| Register        | Value             |
|-----------------|-------------------|
| <b>Current</b>  |                   |
| R0              | 0x00375F00        |
| R1              | 0x001BAF80        |
| R2              | 0x40000000        |
| R3              | 0x00000000        |
| R4              | 0x00000000        |
| R5              | 0x00000000        |
| R6              | 0x00000000        |
| R7              | 0x00000000        |
| R8              | 0x00000000        |
| R9              | 0x00000000        |
| R10             | 0x00000000        |
| R11             | 0x00000000        |
| R12             | 0x00000000        |
| R13 (SP)        | 0x00000000        |
| R14 (LR)        | 0x00000000        |
| <b>R15 (PC)</b> | <b>0x00000070</b> |
| CPSR            | 0x000000D3        |
| SPSR            | 0x00000000        |
| User/System     |                   |

Memory 1

Address: 0x40000000

|             |                                                 |
|-------------|-------------------------------------------------|
| 0x40000000: | 00 5F 37 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| 0x40000015: | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| 0x4000002A: | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| 0x4000003F: | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| 0x40000054: | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |

→ 레지스터 R0 에 10!값이 저장된 것을 확인 할 수 있다.

→ 메모리 주소 0x40000000 에 결과값이 저장된 것을 확인할 수 있다.

### 문제 2번

#### 1. Problem Statement

이 과제의 전반적인 목적은 ARM 어셈블리 기본 명령어의 이해와 활용 능력과, 스택 및 메모리 관리 능력, 재귀 함수 구현 능력, 레지스터와 메모리 조작 능력, 성능 분석 및 최적화 능력을 키우기 위한 과제이다.

**Recursive function을 이용한 팩토리얼 계산(0x4000000에 저장)**

그 중 2번 문제는 재귀 함수와 스택 사용을 통한 팩토리얼 계산을 구현해 내고, 이를 arm 코드로 작성할 수 있는 능력을 키울 수 있다. 재귀함수를 사용하여  $n-1$  하는 로직을 구현하고 이를 mul 하는 과정을 통해 재귀함수와 스택 사용 능력을 키울 수 있다.

#### 2. Method

이 코드를 설계하기 위해선 먼저, 재귀함수와 스택에 대해 생각해야 한다.

재귀함수

먼저 팩토리얼의  $n-1$  항을 구해내는 것을 재귀적으로 해야한다. 그러기 위해서는 종료조건을 설정해야 한다.  $N!$  의 계산에 있어서 종료조건은  $n$ 이 최소 1이상 일 때 이다. 이때를 base case 로 설정한다. 그리고 브랜치 명령어를 사용하여 재귀호출을 실행한다. 재귀호출이 모두 완료되고 반환 될때마다 스택에서 이전 값을 복원한 뒤 mul 을 하여 값을 누적해 나간다.

스택

재귀 호출이 발생하면 현재값과 복귀해야할 주소를 가지고 있는 링크 레지스터를 스택에 저장한다. 그리고 그렇게 저장해 나가다가 pop 명령어를 만나면 가장 최근값부터 복원한다. 모든 결과가 계산 되고 재귀함수가 마무리 되면 결과값을 지정된 주소에 저장하고 svc 0 명령어를 사용하여 프로그램을 종료한다.

이때 주의 해야 할 것은 루프와 명령어 들의 위치이다. Sub 을 어디에서 해줄 것인지, mul을 어디서 해줄 것인지 고민하고 접근해야 한다.

## 3. Result

예상 결과로는 코드를 디버깅 할 때, 레지스터에 담기는 값을 확인해본다면, 재귀 호출 할 때 마다 초기값인 10에서 -1 하면서 1까지 도달 하는 것을 확인할 수 있고, 종료조건을 만나서 복귀할 때마다 이전 값이 복원 되어 누적된 팩토리얼 값이 저장되는 과정을 볼 수 있을 것이다. 따라서 디버깅 할 때 주의깊게 봐야하는 레지스터는 r0,r1, sp 이다. 그리고 마지막으로 내가 지정한 주소인 0x40000000에 10! 값이 저장이 되는 지 확인해야한다.

-재귀 호출 단계에서 sp

각 재귀 호출마다 r0,lr이 스택에 push 되므로 스택 내용이 증가한다. 이때 r0 는 10에서 1로 감소한다. Push 는 stmfd 와 같다. 즉 stmfd sp!,{r0} == push r0 이다.

-복귀 단계에서 스택 복원 및 mul

재귀 호출이 종료된 후 복귀할 때마다 스택에서 이전 r0 값을 r1에 복원하고, MUL r0, r1, r0을 통해 팩토리얼 계산을 누적된다.

-최종 결과 확인

모든 재귀 호출이 끝나면 r0 에는 10! 값 즉, 3628800이 저장된다. 이 값이 메모리 의 0x40000000 번지에 저장되어야한다.

The screenshot displays a debugger interface with three main panels:

- Registers:** A list of registers with their current values. R0 is highlighted, showing 0x00375F00. Other registers like R1, R2, etc., are at 0x00000000.
- Disassembly:** A list of assembly instructions. Line 11: MOV r7, #1 ;Set up system call for program exit. Line 12: MOV r0, #0 ;Exit code 0. Line 13: SVC 0 ;.
- Memory:** A window showing memory contents at address 0x40000000. The value 0x005F3700 is visible at offset 0x00000000.

->레지스터 r0에 10! (10진수로 3628800, 16진수로 375F00이 담긴 것을 확인 할 수 있다. )

-> 메모리 0x40000000 번지에 결과 값이 저장됨을 확인 할 수있다.

-> 마지막으로 시스템 콜들을 이용해 프로그램 종료.

## 문제 3번

### 1. Problem Statement

이 문제의 목적은 ARM에서 곱셈 연산의 성능을 분석하고, 피연산자의 순서가 연산 성능에 어떤 영향을 미치는지 확인하는 것이다.  $17 \times 3$ 과  $3 \times 17$ 의 순서로 연산을 수행하여 처리 시간을 비교하고, 레지스터와 피연산자 순서가 연산 속도에 미치는 영향을 확인한다. 두 연산의 차이가 발생하는 이유와 성능 차이에 영향을 미치는 요인을 분석 한다.

### 2. Method

두 곱셈 연산의 성능을 비교하기 위해 다음과 같은 접근 방식을 사용한다.

-초기 값 설정

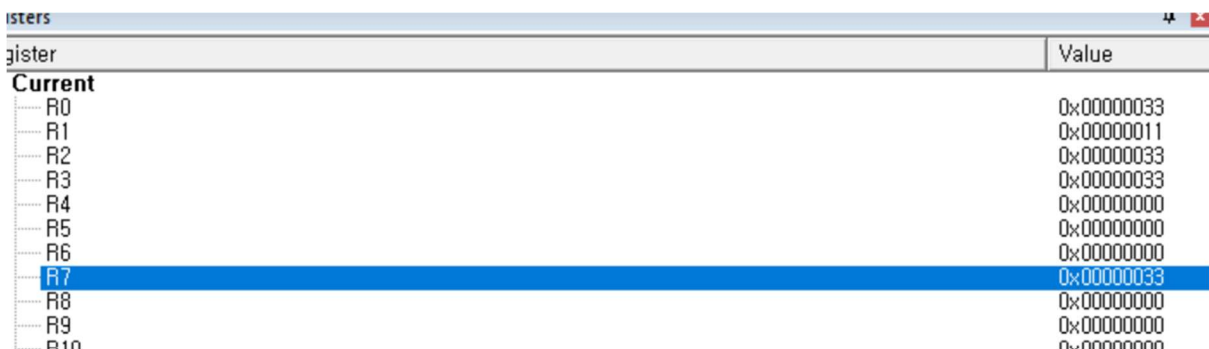
먼저 곱셈에 사용할 값인 17과 3을 각각의 레지스터인 r0와 r1에 설정한다. 이를 통해 두 값의 곱을 비교할 수 있는 준비가 된다.

-곱셈 연산 수행

첫 번째 실험에서는  $17 \times 3$ 을 수행하고, 두 번째 실험에서는  $3 \times 17$ 을 수행한다. 이때 각각의 실험에서 ARM9E-S 아키텍처의 MUL 명령어를 사용하여 피연산자의 순서를 다르게 배치하여 실행한다. 이를 통해 피연산자 순서에 따른 연산 시간 차이가 발생하는지 확인할 수 있다.

### 3. Result

성능을 확인 하기 위해서는 성능 측정 타이머 레지스터를 사용해야 한다. 이론적으로 알아보았는데, 이는 일반적으로 arm 프로세서의 사이클 카운터 Performance Monitoring Unit (PMU) 라는 타이머가 있다. 하지만 이 기능은 arm의 pmu 가 활성화 여부나 하드웨어에 따라 안될 수 도 있다.



| Register | Value      |
|----------|------------|
| Current  |            |
| R0       | 0x00000033 |
| R1       | 0x00000011 |
| R2       | 0x00000033 |
| R3       | 0x00000033 |
| R4       | 0x00000033 |
| R5       | 0x00000000 |
| R6       | 0x00000000 |
| R7       | 0x00000033 |
| R8       | 0x00000000 |
| R9       | 0x00000000 |
| R10      | 0x00000000 |

-> r2 =  $17 \times 3$

->r3 =  $3 \times 17$  를 수행 한 모습이다.

이론적 분석 :

$17 \times 3$ 과  $3 \times 17$ 의 성능 차이를 ARM에서 비교하는 것은 이론적으로 흥미로운 접근이다. 그러나 실제 실행에서는 성능 차이가 나타나지 않을 가능성이 높다. 이와 같은 경우, 발생할 수 있는 이론적인 문제점에 대해 분석해 보겠다. 왜냐면 컴퓨터는 너무 진화했고, 이것은 너무 작은 연산이기 때문이다.

### ARM 아키텍처의 곱셈 연산 최적화

대부분의 ARM 프로세서에서는 MUL 명령어를 사용한 기본 곱셈 연산이 피연산자의 순서와 관계없이 수행된다. 이는 곱셈 연산이 항상 동일한 사이클 수를 사용하도록 보장하기 위한 최적화 설계로, 연산 순서에 따른 성능 차이가 발생하지 않도록 한다.

ARM 아키텍처는 이러한 최적화를 통해 연산이 순서에 상관없이 일관된 성능을 유지하며, 곱셈 연산을 파이프라인 구조에 최적화하여 빠르게 처리할 수 있도록 한다.

### 컴파일러와 하드웨어의 최적화

ARM 프로세서에서는 컴파일러와 하드웨어가 피연산자의 배치를 자동으로 최적화하여, 동일한 연산의 경우 실행 사이클 수가 동일하게 유지되도록 한다. 이는 컴파일러가 코드 최적화를 통해 레지스터 배치와 연산 순서에 관계없이 동일한 성능을 보장하게끔 하기 위한 설계이다.

예를 들어,  $17 \times 3$ 과  $3 \times 17$ 의 연산 순서가 다르더라도, 파이프라인이 동일한 속도로 두 연산을 수행하도록 관리된다. 이러한 최적화로 인해 실제 실행에서는 피연산자의 순서가 성능에 영향을 미치지 않게 된다.

### 레지스터 사용 방식에 따른 차이

ARM 아키텍처의 레지스터 기반 연산에서는 피연산자의 위치가 곱셈 성능에 영향을 주지 않도록 설계되어 있다. MUL 연산은 두 피연산자의 값을 단순히 곱하는 구조로, MUL r2, r0, r1과 MUL r2, r1, r0는 동일한 연산으로 처리된다. 이로 인해 레지스터 배치나 피연산자의 순서가 성능에 영향을 미치지 않는다.

특히 ARM9E-S 아키텍처에서는 내부적으로 파이프라인이 최적화되어 있어 레지스터 위치가 연산 시간에 미치는 영향이 거의 없도록 되어 있다.

### 실제로 차이가 발생하지 않는 이유

ARM9E-S 아키텍처는 곱셈 연산의 순서가 성능에 영향을 주지 않도록 설계되어 있으며, 파이프라인과 하드웨어 최적화가 이를 보장한다. 이러한 설계는 곱셈 연산이 연산 순서에 관계없이 동일한 시간 내에 수행되도록 한다.

따라서  $17 \times 3$ 과  $3 \times 17$ 의 곱셈 연산을 비교할 때, 이론적으로는 순서에 따른 차이가 발생할 가능성이 있지만, ARM9E-S의 최적화 덕분에 실제로는 차이가 나타나지 않게 된다.



### 문제 4번

#### 1. Problem Statement

이 과제의 목적은 ARM 어셈블리에서 스택과 블록 데이터 전송 명령어 (STM, LDM)를 활용하여 레지스터의 값을 교환하는 방법을 이해하는 것이다. 이 과제를 통해 스택을 사용하는 방법과 STMFD / LDMFD 와 같은 블록 데이터 전송 명령어를 활용하여 레지스터 값을 원하는 순서에 맞춰 교환할 수 있는 능력을 기르게 된다

구체적으로, 레지스터 R0부터 R7까지에 각각 1에서 8까지의 값을 할당한 후, 주어진 교환 순서에 따라 레지스터 값을 서로 교환하는 것이다. 이때 단순한 MOV 명령어로 직접 교환하지 않고, 스택이나 블록 데이터 전송 명령어를 사용하여 레지스터 값을 교환하도록 요구된다.

#### 2. Method

-레지스터 초기화:

MOV 명령어를 사용하여 R0부터 R7까지 각각 1에서 8까지의 값을 할당한다. 초기화가 완료되면, 각 레지스터는 다음과 같은 값을 갖게 된다: 각각 r0=1 부터 하나씩 증가시켜 r7=8을 갖도록 초기화

-스택을 이용한 값 저장:

초기화가 완료된 R0부터 R7까지의 값을 스택에 저장하여 임시로 보관한다.

이를 위해 STMFD sp!, {r0-r7} 명령어를 사용하여 R0에서 R7까지의 값들을 스택에 푸시(push)한다. 이 명령어는 블록 전송 명령어다.이렇게 함으로써, 스택에는 R0부터 R7까지의 초기 값이 차례로 저장된다.

- 값 교환:

Figure 1에 제시된 교환 순서에 따라 레지스터의 값을 재배치하기 위해, 스택에서 값을 특정한 순서로 로드한다.

LDMFD fmf 사용하여 스택에 저장된 값을 지정된 순서대로 레지스터에 다시 할당한다.

이때, 스택에서 값을 로드하는 순서를 조정하여 각 레지스터가 Figure 1의 요구대로 값을 갖도록 한다. 예를 들어:

R0의 값이 R1으로 이동해야 한다면, LDMFD 명령어로 스택에서 R1에 값을 로드한다.

이러한 방식으로 모든 레지스터에 대해 지정된 교환 순서에 따라 값을 로드하여 스택을 통해 레지스터 값을 간접적으로 교환하게 된다.

이렇게 블록 데이터 전송을 통해 값을 교환함으로써, 단순한 MOV 명령어로 하나씩 교환하는 것보다 효율적으로 교환을 수행할 수 있다.



### 3. Result

문제 처럼 레지스터가 값이 교환 된다면 아래 와 같이 각각의 레지스터 값이 바뀔것이다.

|    |   |
|----|---|
| r0 | 3 |
| r1 | 1 |
| r2 | 4 |
| r3 | 6 |
| r4 | 7 |
| r5 | 8 |
| r6 | 2 |
| r7 | 5 |

| Registers      |            | Registers |            |
|----------------|------------|-----------|------------|
|                |            | Register  | Value      |
| <b>Current</b> |            | R0        | 0x00000003 |
| R0             | 0x00000001 | R1        | 0x00000001 |
| R1             | 0x00000002 | R2        | 0x00000004 |
| R2             | 0x00000003 | R3        | 0x00000006 |
| R3             | 0x00000004 | R4        | 0x00000007 |
| R4             | 0x00000005 | R5        | 0x00000008 |
| R5             | 0x00000006 | R6        | 0x00000002 |
| R6             | 0x00000007 | R7        | 0x00000005 |
| R7             | 0x00000008 | R8        | 0x00000000 |
|                |            | R9        | 0x00000000 |
|                |            | R10       | 0x00000000 |
|                |            | R11       | 0x00000000 |

➔ 왼쪽은 초기 레지스터 입력값

코드가 시작되면 R0부터 R7까지가 각각 1부터 8까지의 값으로 초기화된다.

➔ 값 교환후 레지스터 입력값

R0에 저장되었던 1이 R1으로 이동하고, R1에 저장된 2가 R6으로 이동하는 식으로 모든 값이 교환된다.

➔ 예상 값과 비교해보면 값이 올바르게 교환된 것을 확인 할 수 있다.

### Discussion and Conclusion

#### -과제를 수행하며 느낀 점

ARM 어셈블리 프로그래밍을 통해 레지스터와 스택, 그리고 블록 데이터 전송 명령어(STM, LDM)의 사용법을 익히는 기회를 가질 수 있었습니다. 특히, 문제를 통해 각 연산의 구조와 최적화 방법, 그리고 데이터 전송과 관련된 ARM 아키텍처의 특징을 이해할 수 있었습니다. 재귀 호출을 사용한 팩토리얼 계산이나 레지스터 값을 스택을 활용해 교환하는 문제는 어셈블리 언어의 기초적인 연산 원리를 체험할 수 있는 좋은 연습이 되었습니다.

#### -개선 방향

기초 구조 학이 필요하다는 것을 느꼈습니다. 어셈블리의 기본적인 명령어 구조에 대해 더 많은 연습이 필요하다고 느꼈습니다. 특히 STM과 LDM 같은 블록 데이터 전송 명령어에 대해 더 깊이 이해하고, 이를 다양한 문제에 응용할 수 있는 능력을 키워야 할 필요성을 느꼈습니다.

#### - 과제 수행 중 발생한 문제점

먼저 INI 파일을 넣지 않아 정말 한참 해맸습니다. 그것이 원인이라고는 생각도 하지 못했습니다. 그리고,STMFD와 LDMFD 명령어를 사용하여 레지스터 값을 교환하는 문제에서, 초기 레지스터 값을 스택에 저장하고 순서에 맞게 다시 불러오는 과정이 헷갈렸습니다. 특히, LDMFD 명령어를 사용할 때 각 레지스터의 값을 불러오는 순서를 정확히 맞추지 않으면, 예상치 못한 값이 다른 레지스터에 할당될 수 있었습니다. 이를 해결하기 위해 문제의 요구 사항을 꼼꼼히 확인하며 테스트를 여러 차례 수행했습니다.

### Reference

<https://stackoverflow.com/questions/53130399/recursive-function-in-arm-assembly-language>

## 명예서약서 (Honor code)

본인은 [어셈블리프로그램설계및실습] 수강에 있어 다음의 명예서약을 준수할 것을 서약합니다.

### 1. 기본 원칙

- 본인은 공학도로서 개인의 품위와 전문성을 지키며 행동하겠습니다.
- 본인은 안전, 건강, 공평성을 수호하는 책임감 있는 공학도가 되겠습니다.
- 본인은 우리대학의 명예를 지키고 신뢰받는 구성원이 되겠습니다.

### 2. 학업 윤리

- 모든 과제와 시험에서 정직하고 성실한 태도로 임하겠습니다.
- 타인의 지적 재산을 존중하고, 표절을 하지 않겠습니다.
- 모든 과제는 처음부터 직접 작성하며, 타인의 결과물을 무단으로 사용하지 않겠습니다.
- 과제 수행 중 받은 도움이나 협력 사항을 보고서에 명시적으로 기재하겠습니다.

### 3. 서약 이행

본인은 위 명예서약의 모든 내용을 이해하였으며, 이를 성실히 이행할 것을 서약합니다.

학번 | 2022202109 | 이름 | 이호정 | 서명 | 이호정